Practical No. 3

AIM

Design and develop an Android application demonstrating Simple Downstream Messaging using Firebase Cloud Messaging (FCM).

OBJECTIVE

To develop an Android application that receives a message sent from **Firebase Cloud Messaging (FCM)** and displays it as a notification on the Android device.

SOFTWARE REQUIREMENTS

- Android Studio
- Java
- Firebase Console
- Firebase Cloud Messaging (FCM)
- Android Emulator / Android device
- Internet connection

THEORY

What is Downstream Messaging?

Downstream messaging is the process of sending a message **from a server/cloud service to an Android device**.

In this practical, Firebase Cloud Messaging is used:

Firebase Console

↓

Firebase Cloud Messaging

↓

Android Application

↓

Notification

The Firebase **registration token** identifies the particular Android app installation/device to which the message is sent.

PROCEDURE

Step 1: Create an Android Studio Project

1. Open **Android Studio**.
2. Click **New Project**.
3. Select **Empty Views Activity**.
4. Click **Next**.
5. Enter the following details:

   Name : SimpleDownstreamMessaging

   Language : Java
6. Select a suitable Minimum SDK.
7. Click **Finish**.
8. Wait for Android Studio to complete the Gradle setup.

Step 2: Create a Firebase Project

1. Open the Firebase Console:

Firebase Console

2. Sign in with a Google account.
3. Click **Create a project**.
4. Enter:

SimpleDownstreamMessaging

5. Google Analytics can be disabled for this practical.
6. Click **Create project**.
7. After the project is created, open the project.

Step 3: Register the Android App with Firebase

1. In the Firebase project, click the **Android icon**.
2. Enter the Android package name.

For this project:

com.example.simpledownstreammessaging

The package name can be checked at the top of MainActivity.java:

package com.example.simpledownstreammessaging;

3. Enter an optional app nickname:

Simple Downstream Messaging

4. Leave SHA-1 empty for this basic practical.
5. Click **Register app**.

Step 4: Download google-services.json

1. Firebase will display the option **Download google-services.json**.
2. Download the file.
3. Place the file inside the **app** folder.

Correct location:

SimpleDownstreamMessaging

```
|
└── app
    |
    └── google-services.json
```

Important

The filename must be exactly:

google-services.json

It should not be:

google-services (1).json

or

google-services.json.json

Step 5: Configure Project-level Gradle

Open:

Gradle Scripts → build.gradle.kts (Project: SimpleDownstreamMessaging)

Add the Google Services plugin:

```
// Top-level build file where you can add configuration options common to all sub-projects/modules.


plugins {
   alias(libs.plugins.android.application) apply false
```

```
    id("com.google.gms.google-services") version "4.5.0" apply false
}
```

Step 6: Configure App-level Gradle

Open:

Gradle Scripts → build.gradle.kts (Module: app)

Use the following configuration:

```
plugins {
    alias(libs.plugins.android.application)
    id("com.google.gms.google-services")
}

android {
    namespace = "com.example.simpledownstreammessaging"

    compileSdk {
        version = release(36) {
            minorApiLevel = 1
        }
    }

    defaultConfig {
        applicationId = "com.example.simpledownstreammessaging"
        minSdk = 24
        targetSdk = 36
        versionCode = 1
        versionName = "1.0"

        testInstrumentationRunner = "androidx.test.runner.AndroidJUnitRunner"
    }
```

```
    buildTypes {
        release {
            optimization {
                enable = false
            }
        }
    }

    compileOptions {
        sourceCompatibility = JavaVersion.VERSION_11
        targetCompatibility = JavaVersion.VERSION_11
    }
}

dependencies {

    implementation(platform("com.google.firebase:firebase-bom:34.18.0"))
    implementation("com.google.firebase:firebase-messaging")

    implementation(libs.activity.ktx)
    implementation(libs.appcompat)
    implementation(libs.constraintlayout)
    implementation(libs.material)

    testImplementation(libs.junit)
    androidTestImplementation(libs.espresso.core)
    androidTestImplementation(libs.ext.junit)
}
```

After adding the code, click **Sync Now**.

Step 7: Create the Main Layout

Open:

app → res → layout → activity_main.xml

Paste:

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
   xmlns:android="http://schemas.android.com/apk/res/android"
   xmlns:app="http://schemas.android.com/apk/res-auto"
   xmlns:tools="http://schemas.android.com/tools"
   android:id="@+id/main"
   android:layout_width="match_parent"
   android:layout_height="match_parent"
   tools:context=".MainActivity">

   <TextView
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:text="Hello World!"
      app:layout_constraintBottom_toBottomOf="parent"
      app:layout_constraintEnd_toEndOf="parent"
      app:layout_constraintStart_toStartOf="parent"
      app:layout_constraintTop_toTopOf="parent" />

</androidx.constraintlayout.widget.ConstraintLayout>
```

This creates the basic screen of the application.

Step 8: Configure MainActivity.java

Open:

app → java → com.example.simpledownstreammessaging → MainActivity.java

Replace the existing code with:

```
package com.example.simpledownstreammessaging;
```

```
import android.Manifest;
import android.content.pm.PackageManager;
import android.os.Build;
import android.os.Bundle;
import android.util.Log;

import androidx.activity.EdgeToEdge;
import androidx.activity.result.ActivityResultLauncher;
import androidx.activity.result.contract.ActivityResultContracts;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.content.ContextCompat;
import androidx.core.graphics.Insets;
import androidx.core.view.ViewCompat;
import androidx.core.view.WindowInsetsCompat;

import com.google.firebase.messaging.FirebaseMessaging;

public class MainActivity extends AppCompatActivity {

   private final ActivityResultLauncher<String> requestPermissionLauncher =
         registerForActivityResult(
               new ActivityResultContracts.RequestPermission(),
               isGranted -> {
                  if (isGranted) {
                     Log.d("FCM", "Notification permission granted");
                  } else {
                     Log.d("FCM", "Notification permission denied");
                  }
               }
         );
```

```
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);

    EdgeToEdge.enable(this);
    setContentView(R.layout.activity_main);

    // Ask for notification permission on Android 13+
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) {

        if (ContextCompat.checkSelfPermission(
                this,
                Manifest.permission.POST_NOTIFICATIONS
        ) != PackageManager.PERMISSION_GRANTED) {

            requestPermissionLauncher.launch(
                    Manifest.permission.POST_NOTIFICATIONS
            );
        }
    }

    // Get Firebase registration token
    FirebaseMessaging.getInstance().getToken()
            .addOnCompleteListener(task -> {

                if (!task.isSuccessful()) {
                    Log.e(
                            "FCM",
                            "Failed to get token",
                            task.getException()
                    );
```

```
                return;
            }

            String token = task.getResult();

            Log.d("FCM_TOKEN", token);
        });

    ViewCompat.setOnApplyWindowInsetsListener(
        findViewById(R.id.main),
        (v, insets) -> {

            Insets systemBars =
                insets.getInsets(
                    WindowInsetsCompat.Type.systemBars()
                );

            v.setPadding(
                systemBars.left,
                systemBars.top,
                systemBars.right,
                systemBars.bottom
            );

            return insets;
        });
  }
}
```

Purpose of this code

The code:

- Requests notification permission.

- Connects the application with Firebase Messaging.
- Generates the FCM registration token.
- Displays the token in Logcat.

Step 9: Create MyFirebaseMessagingService.java

Create the Java class

Go to:

app → java → com.example.simpledownstreammessaging

Right-click the package and select:

New → Java Class

Enter:

MyFirebaseMessagingService

Open the newly created file and paste:

```
package com.example.simpledownstreammessaging;

import android.app.NotificationChannel;
import android.app.NotificationManager;
import android.os.Build;

import androidx.annotation.NonNull;
import androidx.core.app.NotificationCompat;

import com.google.firebase.messaging.FirebaseMessagingService;
import com.google.firebase.messaging.RemoteMessage;

public class MyFirebaseMessagingService extends FirebaseMessagingService {

    @Override
    public void onMessageReceived(@NonNull RemoteMessage remoteMessage) {

        String title = "Firebase Message";
```

```
    String message = "New message received";

    if (remoteMessage.getNotification() != null) {

        if (remoteMessage.getNotification().getTitle() != null) {
            title = remoteMessage.getNotification().getTitle();
        }

        if (remoteMessage.getNotification().getBody() != null) {
            message = remoteMessage.getNotification().getBody();
        }
    }

    showNotification(title, message);
}

private void showNotification(String title, String message) {

    String channelId = "my_channel";

    NotificationManager manager =
            (NotificationManager) getSystemService(NOTIFICATION_SERVICE);

    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {

        NotificationChannel channel = new NotificationChannel(
                channelId,
                "Firebase Messages",
                NotificationManager.IMPORTANCE_HIGH
        );
```

```
            manager.createNotificationChannel(channel);
        }

        NotificationCompat.Builder builder =
                new NotificationCompat.Builder(this, channelId)
                        .setSmallIcon(android.R.drawable.ic_dialog_info)
                        .setContentTitle(title)
                        .setContentText(message)
                        .setPriority(NotificationCompat.PRIORITY_HIGH)
                        .setAutoCancel(true);

        manager.notify(1, builder.build());
    }
}
```

Purpose

FirebaseMessagingService receives the Firebase message.

The method:

onMessageReceived()

handles the received message.

The notification is created using:

NotificationCompat.Builder

Step 10: Modify AndroidManifest.xml

Open:

app → manifests → AndroidManifest.xml

Use:

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

    <uses-permission android:name="android.permission.POST_NOTIFICATIONS" />
```

```
<application
    android:allowBackup="true"
    android:dataExtractionRules="@xml/data_extraction_rules"
    android:fullBackupContent="@xml/backup_rules"
    android:icon="@mipmap/ic_launcher"
    android:label="@string/app_name"
    android:roundIcon="@mipmap/ic_launcher_round"
    android:supportsRtl="true"
    android:theme="@style/Theme.SimpleDownstreamMessaging">

    <service
        android:name=".MyFirebaseMessagingService"
        android:exported="false">

        <intent-filter>
            <action android:name="com.google.firebase.MESSAGING_EVENT" />
        </intent-filter>

    </service>

    <activity
        android:name=".MainActivity"
        android:exported="true"
        android:windowSoftInputMode="adjustResize">

        <intent-filter>
            <action android:name="android.intent.action.MAIN" />

            <category android:name="android.intent.category.LAUNCHER" />
        </intent-filter>
```

```
        </activity>

    </application>

</manifest>
```

Important points

This permission:

```
<uses-permission android:name="android.permission.POST_NOTIFICATIONS" />
```

allows the application to request notification permission on newer Android versions.

This service:

```
<service
    android:name=".MyFirebaseMessagingService"
    android:exported="false">
```

registers the Firebase messaging service with the application.

Step 11: Run the Application

1. Start the Android Emulator.
2. We used a **Pixel 7 emulator**.
3. Click **Run** ▶ in Android Studio.
4. Wait for the application to launch.

The application displays:

Hello World!

This is the default screen of the application.

Step 12: Allow Notification Permission

When the application starts, Android may display:

Allow SimpleDownstreamMessaging to send you notifications?

Select:

Allow ✓

This permission is required for displaying notifications on newer Android versions.

Step 13: Obtain FCM Registration Token

1. Open Android Studio.
2. Select:

View → Tool Windows → Logcat

3. Search for:

FCM_TOKEN

You should see something similar to:

FCM_TOKEN: eECdtn6xT0yX-f_ERYhsmd:APA91...

Copy the complete token.

Note

The token is generated by Firebase and may change if the application is reinstalled.

Step 14: Create a Downstream Message

Open the Firebase Console.

Go to:

Messaging → Create campaign / Notification

Enter:

Notification title

Test Message

Notification text

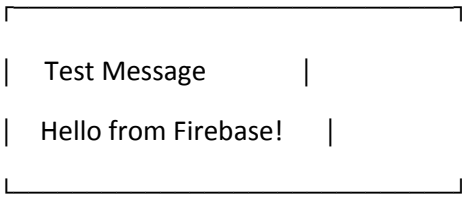
Hello from Firebase!

Step 15: Send the Test Message

1. Select **Send test message**.
2. Paste the **FCM registration token** obtained from Logcat.
3. Add/select the token.
4. Click **Test / Send**.
5. Put the Android application in the background by pressing the **Home** button.
6. Wait for the notification.

Step 16: Observe the Output

The Android device/emulator receives:



The notification can also be viewed by swiping down from the top of the Android emulator.

WORKING

The working of the application can be summarized as:

1. Android App starts

↓

2. Firebase generates FCM Token

↓

3. Token identifies the app/device

↓

4. Firebase Console sends message

↓

5. Firebase Cloud Messaging delivers it

↓

6. Android receives the message

↓

7. Notification is created

↓

8. User sees the notification

RESULT

The Android application was successfully designed and developed to demonstrate Simple Downstream Messaging using Firebase Cloud Messaging (FCM). A message was sent from Firebase Console to the Android application and received as a notification on the Android emulator.

VIVA QUESTIONS

1. What is downstream messaging?

Downstream messaging is the process of sending a message from a server/cloud service to a client device.

2. What is FCM?

FCM stands for **Firebase Cloud Messaging**. It is a Firebase service used to send messages and notifications to applications.

3. What is an FCM registration token?

It is a unique token used to identify a particular app installation/device for receiving Firebase messages.

4. Which class is used to receive Firebase messages?

FirebaseMessagingService

5. Which method handles the received message?

onMessageReceived()

6. What is the direction of downstream messaging?

Firebase/Server → Android Device

7. What is google-services.json?

It is the Firebase configuration file that connects the Android application with the Firebase project.

8. Why is notification permission required?

Newer Android versions require the application to obtain the user's permission before displaying notifications.

9. What is a notification channel?

A notification channel is an Android mechanism used to organize and control notifications.

10. What is the difference between downstream and upstream messaging?

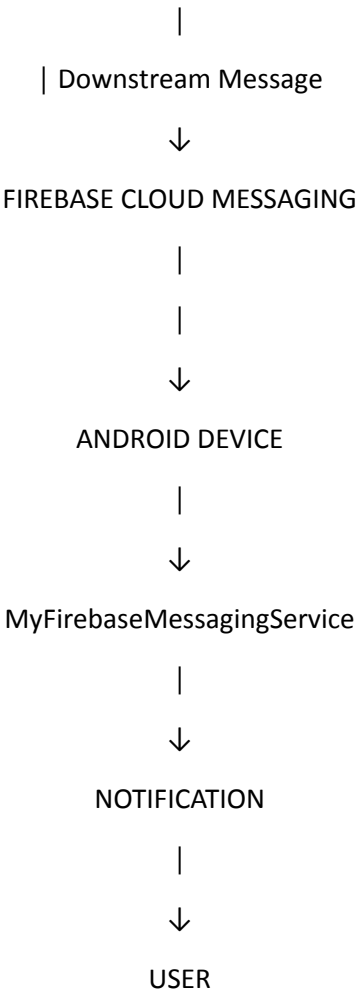
Downstream	**Upstream**
Server → Device	Device → Server
Sends message to application	Sends message from application
Example: Firebase notification	Example: Application sending data

FLOW DIAGRAM

You can draw this in your journal:

FIREBASE CONSOLE

|



Files used in the practical

File	**Purpose**
activity_main.xml	Designs the Android screen
MainActivity.java	Requests permission and gets FCM token
MyFirebaseMessagingService.java	Receives and displays Firebase messages
AndroidManifest.xml	Declares notification permission and Firebase service
build.gradle.kts	Adds/configures Firebase dependencies
google-services.json	Connects the Android app to Firebase